

Bài toán: bài dễ nhất

Nhóm 2: Nguyễn Trường Giang - Nguyễn Ngọc Hưng

Ngày 19 tháng 11 năm 2025

1 Mô tả bài toán

Cho kích thước khung hình chữ nhật tổng thể có chiều rộng W và chiều cao H . Có n hình chữ nhật con, hình chữ nhật thứ i được mô tả bởi hai đỉnh đối diện

$$(x_{1i}, y_{1i}), \quad (x_{2i}, y_{2i})$$

(với giả thiết $x_{1i} \leq x_{2i}$, $y_{1i} \leq y_{2i}$ sau khi chuẩn hoá).

Có thêm m điểm, điểm thứ j có tọa độ

$$(p_j^x, p_j^y)$$

kèm theo một giá trị “điểm” t_j . Mỗi điểm sẽ được cộng vào đúng một nơi:

- Nếu điểm nằm trong một (hoặc nhiều) hình chữ nhật, điểm đó được cộng cho **hình chữ nhật “phủ” nó gần nhất theo trục x tại thời điểm quét** (chi tiết ở phần thuật toán).
- Nếu điểm nằm ngoài mọi hình chữ nhật, điểm đó được cộng vào phần *ngoài vùng* (chỉ số 0).

Yêu cầu: tính tổng điểm nhận được của từng hình chữ nhật (n giá trị) và tổng điểm của tất cả các điểm không thuộc hình chữ nhật nào (giá trị thứ $n + 1$).

2 Ý tưởng thiết kế thuật toán tổng quát

Kí hiệu:

- n : số hình chữ nhật.
- m : số điểm.

Ý tưởng chính:

1. **Nén tọa độ theo trục y** : vì chỉ các biên y_{1i}, y_{2i} mới làm thay đổi tập hình chữ nhật đang “phủ”. Ta gom tất cả các giá trị y_{1i}, y_{2i} , sắp xếp tăng dần, loại trùng để tạo mảng Y . Mỗi khoảng $[Y_k, Y_{k+1})$ tương ứng với một *segment* trong segment tree.
2. **Quét theo trục x (sweep line)**: xây dựng các sự kiện:

- Khi bắt đầu đi vào hình chữ nhật i (tại x_{1i}): *thêm* hình chữ nhật i vào tất cả các segment $[L_i, R_i)$ tương ứng với khoảng $[y_{1i}, y_{2i})$.
- Khi đi ra khỏi hình chữ nhật i (tại x_{2i}): *loại* hình chữ nhật i khỏi các segment $[L_i, R_i)$.
- Với mỗi điểm (p_j^x, p_j^y) : tạo một sự kiện *truy vấn* tại $x = p_j^x$.

Sắp xếp tất cả các sự kiện theo x tăng dần, nếu trùng x thì xử lý theo thứ tự: *loại* $\rightarrow$ *thêm* $\rightarrow$ *truy vấn*.

3. Segment tree với stack tại mỗi node:

- Mỗi node trong segment tree tương ứng với một khoảng các segment y liên tiếp.
- Tại một node, ta lưu một **stack** các chỉ số hình chữ nhật đang phủ hoàn toàn khoảng đó, theo thứ tự xuất hiện dọc theo trục x .
- Khi “thêm” hình chữ nhật trên đoạn $[L_i, R_i)$, ta đi trên cây segment và *push* chỉ số i vào các node liên quan.
- Khi “loại” hình chữ nhật, ta *pop* ở đúng những node tương ứng.
- Để trả lời truy vấn tại một điểm có chỉ số segment k , ta đi từ lá ứng với k đi lên gốc; node đầu tiên gặp có stack không rỗng, lấy phần tử trên cùng của stack chính là hình chữ nhật “phủ” điểm đó.

Ý tưởng này giúp cập nhật đoạn và truy vấn điểm đều trong thời gian $O(\log n)$.

3 Phân tích 3 subtask

3.1 Subtask 1: Duyệt (brute force)

Phương pháp thiết kế thuật toán:

Brute force + kiểm tra điều kiện hình học đơn giản.

- Ở subtask 1 giả sử các giới hạn rất nhỏ (ví dụ $n, m \leq 2,000$), ta có thể duyệt tất cả các cặp (*hình chữ nhật, điểm*).
- Cách làm này dễ cài đặt, đúng trực tiếp theo định nghĩa: với mỗi điểm, kiểm tra nó nằm trong hình chữ nhật nào rồi cộng điểm tương ứng.
- Không cần cấu trúc dữ liệu phức tạp, phù hợp khi số lượng đối tượng ít.

Độ phức tạp:

- Với mỗi điểm (tổng cộng m điểm) ta duyệt tối đa n hình chữ nhật để kiểm tra bao phủ.
- Độ phức tạp thời gian:

$$O(n \cdot m).$$

- Độ phức tạp bộ nhớ: chỉ lưu danh sách n hình chữ nhật và m điểm, nên $O(n + m)$.

3.2 Subtask 2: Sweep line + nén toạ độ y

Phương pháp thiết kế thuật toán:

Thiết kế thuật toán bằng kỹ thuật quét theo trục (sweep line) kết hợp nén toạ độ theo trục y . Ta vẫn dùng một cấu trúc mảng/DS đơn giản để lưu trạng thái “đang phủ” theo từng đoạn y .

- Khi n, m lớn hơn (ví dụ tới 10^5) thì $O(nm)$ là không dùng được.
- Sweep line theo x giúp biến bài toán 2D thành chuỗi trạng thái 1D theo y ở mỗi cột x .
- Nén toạ độ y làm giảm số lượng đoạn cần quản lý, chỉ còn khoảng $O(n)$ đoạn.
- Sử dụng mảng hoặc vector để lưu “id hình chữ nhật đang phủ” cho từng đoạn y là đủ cho subtask 2, dù việc cập nhật đoạn vẫn còn chậm.

Độ phức tạp:

- Có khoảng $O(n)$ đoạn y sau khi nén.
- Mỗi lần thêm hoặc xoá một hình chữ nhật, ta phải cập nhật trên $O(\text{chiều cao theo } y)$ đoạn.
- Với $2n$ sự kiện hình chữ nhật và m điểm, độ phức tạp ước lượng:

$$O((n + m) \cdot H)$$

với H là số đoạn y (tối đa khoảng $2n$).

- Bộ nhớ: lưu trạng thái cho từng đoạn $y \Rightarrow O(n)$.

3.3 Subtask 3: Sweep line + segment tree

Phương pháp thiết kế thuật toán:

- Vẫn dùng sweep line theo trục x và nén toạ độ theo trục y .
- Thay vì cập nhật từng đoạn y bằng mảng, ta dùng **segment tree thửa**: mỗi node lưu một **stack** các hình chữ nhật đang phủ toàn bộ đoạn đó.
- Khi thêm hình chữ nhật trên khoảng $[L_i, R_i)$, ta thực hiện cập nhật đoạn trên cây (push vào một số node). Khi xoá thì pop ra.
- Truy vấn một điểm tương ứng một **truy vấn điểm trên segment tree**: đi từ lá lên gốc và lấy phần tử trên cùng của stack đầu tiên gặp được.
- Cập nhật đoạn + truy vấn điểm trên segment tree chỉ tốn $O(\log n)$.
- Việc dùng cấu trúc **thửa** (dictionary lưu những node có stack khác rỗng) giúp tiết kiệm bộ nhớ khi số đoạn y lớn nhưng không phải node nào cũng được sử dụng.
- Kỹ thuật stack tại mỗi node đảm bảo khi quét từ trái sang phải, hình chữ nhật “mới nhất” phủ đoạn y sẽ được ưu tiên (phù hợp với yêu cầu bài toán).

Độ phức tạp:

- Nén toạ độ: sắp xếp các giá trị y , độ phức tạp

$$O(n \log n).$$

- Số sự kiện tổng cộng là $2n + m$.
- Mỗi lần *thêm/xóa* hình chữ nhật: cập nhật đoạn trên segment tree $\Rightarrow O(\log n)$.
- Mỗi truy vấn điểm: đi từ lá lên gốc, dừng ở node đầu có stack khác rỗng $\Rightarrow O(\log n)$.

- Tổng thời gian:

$$O((n + m) \log n).$$

- Bộ nhớ:

- Lưu danh sách hình chữ nhật, điểm và mảng nén toạ độ Y : $O(n + m)$.
- Segment tree với stack tại các node được dùng: xấp xỉ $O(n \log n)$ trong trường hợp xấu.

4 Code Python

```
1 import sys
2 from bisect import bisect_left, bisect_right
3
4 class SparseSegStacks:
5     __slots__ = ("n", "size", "stacks")
6     def __init__(self, n_segments: int):
7         self.n = n_segments
8         size = 1
9         while size < n_segments:
10             size <= 1
11             self.size = size
12             self.stacks = {}
13
14     def push_range(self, l: int, r: int, rid: int):
15         if l >= r or self.n == 0:
16             return
17         size = self.size
18         stacks = self.stacks
19         l += size; r += size
20         while l < r:
21             if l & 1:
22                 st = stacks.get(l)
23                 if st is None:
24                     stacks[l] = [rid]
25                 else:
26                     st.append(rid)
27             l += 1
```

```

28         if r & 1:
29             r -= 1
30             st = stacks.get(r)
31             if st is None:
32                 stacks[r] = [rid]
33             else:
34                 st.append(rid)
35         l >>= 1; r >>= 1
36
37     def pop_range(self, l: int, r: int):
38         if l >= r or self.n == 0:
39             return
40         size = self.size
41         stacks = self.stacks
42         l += size; r += size
43         while l < r:
44             if l & 1:
45                 st = stacks.get(l); st.pop()
46                 if not st: del stacks[l]
47                 l += 1
48             if r & 1:
49                 r -= 1
50                 st = stacks.get(r); st.pop()
51                 if not st: del stacks[r]
52             l >>= 1; r >>= 1
53
54     def query_index(self, idx: int):
55         if idx < 0 or idx >= self.n:
56             return None
57         i = idx + self.size
58         stacks = self.stacks
59         while i:
60             st = stacks.get(i)
61             if st:
62                 return st[-1]
63             i >>= 1
64         return None
65
66 def main():
67     data = list(map(int, sys.stdin.buffer.read().split()))
68     it = iter(data)
69     try:
70         W = next(it); H = next(it); n = next(it); m = next(it)
71     except StopIteration:
72         return
73
74     x1 = [0]*n; y1 = [0]*n; x2 = [0]*n; y2 = [0]*n
75     Ys = []
76     for i in range(n):
77         a = next(it); b = next(it); c = next(it); d = next(it)
78         if a > c: a, c = c, a

```

```

79         if b > d: b, d = d, b
80         x1[i], y1[i], x2[i], y2[i] = a, b, c, d
81         Ys.append(b); Ys.append(d)
82
83     px = [0]*m; py = [0]*m; pt = [0]*m
84     for j in range(m):
85         px[j] = next(it); py[j] = next(it); pt[j] = next(it)
86
87     Ys.sort()
88     k = 1
89     for i in range(1, len(Ys)):
90         if Ys[i] != Ys[k-1]:
91             Ys[k] = Ys[i]; k += 1
92     Ys = Ys[:k]
93     Nseg = max(0, len(Ys) - 1)
94     seg = SparseSegStacks(Nseg)
95
96     L = [0]*n; R = [0]*n
97     for i in range(n):
98         L[i] = bisect_left(Ys, y1[i])
99         R[i] = bisect_left(Ys, y2[i])
100
101     leaf_idx = [0]*m
102     for j in range(m):
103         kj = bisect_right(Ys, py[j]) - 1
104         leaf_idx[j] = kj
105
106     events = []
107     events_extend = events.extend
108     for i in range(n):
109         h = R[i] - L[i]
110         events_extend([(x2[i], 0, h, i),
111                        (x1[i], 1, -h, i)])
112     for j in range(m):
113         events.append((px[j], 2, 0, j))
114     events.sort()
115
116     ans = [0]*(n+1)
117
118     for x, typ, key, idx in events:
119         if typ == 0:
120             seg.pop_range(L[idx], R[idx])
121         elif typ == 1:
122             seg.push_range(L[idx], R[idx], idx)
123         else:
124             kj = leaf_idx[idx]
125             t = pt[idx]
126             if kj < 0 or kj >= Nseg:
127                 ans[-1] += t
128             else:
129                 rid = seg.query_index(kj)

```

```
130         if rid is None:
131             ans[-1] += t
132         else:
133             ans[rid] += t
134
135     out = sys.stdout.write
136     out(" ".join(map(str, ans)))
137
138 if __name__ == "__main__":
139     main()
```